

Laporan Praktikum Minggu Ke-5 Kontrol Cerdas

Minggu ke-5

Nama : Putu Sriwahyuningsih
NIM : 224308066
Kelas : TKA 7C
Akun Github : <https://github.com/putusriwahyu>

1. Judul Percobaan

Deep Reinforcement Learning untuk Kontrol Kompleks

2. Tujuan Percobaan

- Memahami konsep **Deep Reinforcement Learning (DRL)** dalam kontrol sistem kompleks.
- Mengimplementasikan **Deep Q-Network (DQN)** untuk kontrol otomatis.
- Menganalisis performa DRL dibandingkan dengan metode kontrol konvensional.

3. Landasan Teori

Deep Reinforcement Learning (DRL) adalah pengembangan dari *Reinforcement Learning* (RL) yang memadukan pembelajaran berbasis pengalaman dengan jaringan saraf tiruan. Jika RL tradisional menggunakan tabel untuk mencatat semua kemungkinan situasi dan tindakan, DRL menggunakan jaringan saraf agar mampu menangani masalah yang lebih kompleks dan dinamis. Pendekatan ini membuat DRL banyak digunakan dalam bidang seperti robotika, kendaraan otonom, dan sistem tenaga listrik karena mampu belajar langsung dari pengalaman tanpa memerlukan model matematis yang rumit (Mnih et al., 2015)

Salah satu algoritma utama dalam DRL adalah **Deep Q-Network (DQN)**. DQN memungkinkan sistem untuk mempelajari keputusan terbaik berdasarkan hasil dari tindakan-tindakan sebelumnya. Dengan memanfaatkan jaringan saraf, DQN dapat mengenali pola dari data dan menyesuaikan tindakannya agar hasilnya semakin baik. Selain itu, DQN menggunakan teknik seperti *experience replay* dan *target network* untuk menjaga agar proses pembelajaran tetap stabil

(Mnih et al., 2013).

Dalam sistem kontrol, DRL dianggap lebih fleksibel dibandingkan metode konvensional seperti PID atau Fuzzy Logic. DRL mampu beradaptasi dengan perubahan lingkungan secara otomatis, sedangkan metode klasik biasanya memerlukan penyesuaian manual. Menurut (Li, 2018), keunggulan utama DRL terletak pada kemampuannya menyesuaikan strategi kontrol secara terus-menerus berdasarkan pengalaman baru. Meski begitu, DRL juga memiliki tantangan, seperti kebutuhan waktu pelatihan dan sumber daya komputasi yang tinggi. Namun, dengan perkembangan teknologi saat ini, metode ini semakin banyak diterapkan untuk menciptakan sistem kontrol yang cerdas dan adaptif di berbagai bidang, mulai dari robotika hingga efisiensi energi.

Secara keseluruhan, *Deep Reinforcement Learning*, khususnya algoritma *Deep Q-Network*, memberikan pendekatan baru dalam dunia sistem kontrol modern. Dengan kemampuan untuk belajar dari pengalaman dan beradaptasi terhadap perubahan lingkungan, metode ini membuka peluang besar dalam pengembangan sistem kontrol cerdas di berbagai bidang, mulai dari otomasi industri, robotika, hingga pengelolaan energi yang efisien.

4. Analisis dan Diskusi

a. Diskusi

1. Apa kelebihan dan kelemahan DQN dibandingkan metode Q-Learning klasik?

Kelebihan:

DQN bisa menangani masalah yang lebih rumit dan besar daripada Q-Learning biasa. Kalau Q-Learning klasik cuma bisa dipakai untuk lingkungan sederhana yang semua kemungkinannya bisa dicatat di tabel, DQN memakai jaringan saraf agar bisa “mengira-ngira” keputusan terbaik dari data yang sangat banyak. Jadi, DQN bisa digunakan pada sistem yang punya banyak variabel atau kondisi yang terus berubah, seperti robot, mobil otonom, atau permainan kompleks. Selain itu, DQN juga belajar lebih cepat karena bisa mengenali pola dari pengalaman sebelumnya.

Kelemahan:

DQN butuh waktu pelatihan yang lama dan memerlukan komputer dengan kemampuan tinggi (misalnya GPU) karena prosesnya melibatkan jaringan saraf. Selain itu, hasil pelatihannya tidak selalu stabil—kadang bisa “bingung” jika datanya tidak cukup banyak atau pengaturannya tidak tepat. Sementara itu, Q-Learning klasik lebih sederhana, mudah dipahami, dan cepat digunakan untuk masalah kecil.

2. Bagaimana cara mengadaptasi DQN untuk kontrol sistem dinamis seperti drone atau robot industri?

Untuk menerapkan DQN pada sistem dinamis seperti drone atau robot industri, langkah utamanya adalah membuat sistem belajar dari pengalaman melalui percobaan berulang. Dalam proses ini, sistem akan diberi “hadiah” ketika berhasil melakukan tindakan yang benar, seperti menjaga keseimbangan atau menghindari rintangan, dan “hukuman” ketika gagal. Seiring waktu, DQN akan belajar tindakan mana yang paling efektif dalam berbagai situasi. Agar lebih realistis dan aman, pembelajaran biasanya dimulai dari simulasi komputer sebelum diterapkan pada perangkat fisik. Data dari sensor seperti posisi, kecepatan, atau kemiringan digunakan sebagai masukan untuk jaringan saraf agar sistem dapat mengenali kondisi lingkungannya secara akurat. Selain itu, DQN dapat dibuat adaptif dengan terus memperbarui modelnya selama digunakan, sehingga mampu menyesuaikan diri dengan perubahan kondisi lingkungan atau beban kerja robot. Dengan cara ini, DQN dapat membantu sistem seperti drone atau robot industri menjadi lebih cerdas, stabil, dan mampu mengambil keputusan sendiri dalam menghadapi situasi yang dinamis.

5. Assignment

Pada tugas minggu ini, saya membuat dan memodifikasi program Deep Q-Network (DQN) untuk dua jenis environment, yaitu CartPole-v1 dan LunarLander-v2 dari OpenAI Gym. Tujuan tugas ini adalah agar agen bisa belajar mengambil keputusan secara otomatis lewat proses *trial and error*

menggunakan konsep Deep Reinforcement Learning (DRL). Dalam pengerjaannya, saya mempelajari dulu cara kerja dasar DQN yang menggabungkan metode *Q-learning* dengan jaringan saraf tiruan agar agen bisa memperkirakan nilai terbaik dari setiap aksi yang mungkin dilakukan. Setelah itu, saya modifikasi kodenya supaya bisa berjalan di versi TensorFlow dan Gym yang baru, serta menambahkan beberapa teknik tambahan agar pelatihan lebih stabil.

Beberapa teknik yang saya tambahkan antara lain Replay Memory, supaya data pengalaman agen disimpan dan digunakan ulang saat pelatihan agar pembelajaran tidak mudah goyah; Epsilon-Greedy, untuk menyeimbangkan antara eksplorasi dan eksploitasi, sehingga agen tidak hanya mengulang aksi lama tapi juga mencoba hal baru; dan Target Network, supaya pembaruan bobot jaringan tidak terlalu sering berubah drastis sehingga hasilnya lebih stabil. Program ini dibuat menggunakan Python dengan bantuan TensorFlow dan Keras untuk membangun model jaringan saraf, serta Matplotlib untuk menampilkan grafik hasil latihannya.

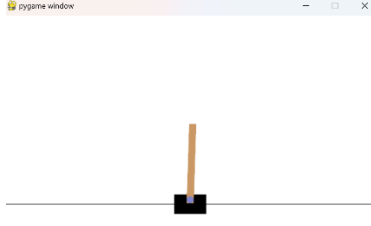
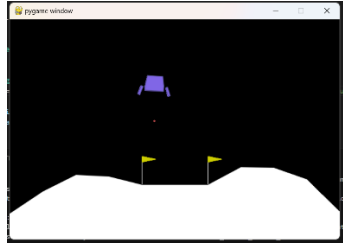
Secara umum, struktur programnya terdiri dari beberapa bagian utama. Bagian awal digunakan untuk menginisialisasi environment, kemudian ada pembuatan model DQN yang terdiri dari beberapa *Dense layer* dengan aktivasi *ReLU*. Setelah itu ada fungsi *select_action()* untuk menentukan aksi berdasarkan strategi epsilon-greedy, dan bagian utama berupa loop pelatihan di mana agen mengambil aksi, menghitung reward, menyimpan pengalaman ke memori, lalu memperbarui jaringan sarafnya. Di akhir, hasilnya divisualisasikan lewat grafik *reward per episode* supaya perkembangan pelatihan bisa terlihat dengan jelas.

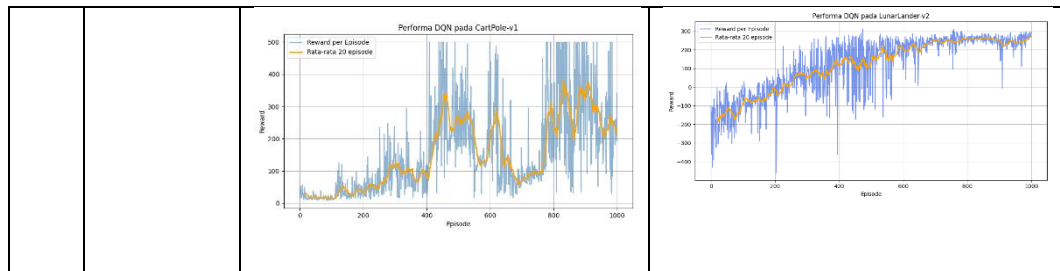
Dari hasil percobaan, pada CartPole-v1 agen bisa belajar dengan cukup cepat. Nilai *reward* meningkat dan stabil setelah sekitar 300 episode, menandakan agen sudah bisa menjaga keseimbangan tiang dengan baik. Sementara itu, pada LunarLander-v2, proses pelatihan berlangsung lebih lama karena kontrol pendaratannya lebih kompleks. Agen butuh waktu lebih banyak untuk belajar mendarat dengan lembut di area target, tapi setelah sekitar 800 episode performanya meningkat dan reward-nya semakin baik. Secara

keseluruhan, tugas ini menunjukkan bahwa DQN cukup efektif untuk membuat agen belajar dan beradaptasi sendiri, meskipun membutuhkan waktu latihan yang cukup panjang dan komputasi yang lumayan besar.

6. Data dan Output Hasil Pengamatan

No	Variabel	Hasil Pengamatan	
		CartPole-v1	Lunar Lander
1.	Waktu Training	CartPole lebih cepat mencapai performa bagus hanya dalam beberapa ratus episode.	LunarLander butuh episode lebih banyak agar hasilnya stabil.
2.	Jumlah Episode yang di training	Cartpole ditraining hingga 1000 episode <pre># Pilih environment (CartPole-v1) env = gym.make("CartPole-v1") state_size = env.observation_space.shape[0] action_size = env.action_space.n agent = DQNAgent(state_size, action_size) episodes = 1000 batch_size = 32</pre>	LunarLander ditraining hingga 1000 episode <pre>EPSILON_DECAY = 0.995 TARGET_UPDATE_EVERY = 1000 MAX_EPISODES = 1000 MAX_STEPS = 1000</pre>
3.	Reward Awal	Reward awal rendah (sekitar 10–30) karena agen belum belajar menjaga keseimbangan.	Reward awal negatif (–200 s.d –150) karena agen sering jatuh sebelum mendarat.
4.	Reward Akhir Maksimum	Reward maksimum mencapai ± 200 setelah sekitar 300 episode.	Reward maksimum mencapai ± 250 setelah ± 800 episode.
5.	Epsilon	Nilai epsilon (ϵ) menurun seiring dengan proses agen belajar <pre>Epsilon = 0.069 Epsilon = 0.069 Epsilon = 0.068 Epsilon = 0.068 Epsilon = 0.068 Epsilon = 0.067 Epsilon = 0.067 Epsilon = 0.067 Epsilon = 0.066 Epsilon = 0.066 Epsilon = 0.066</pre>	Nilai epsilon (ϵ) menurun seiring dengan proses agen belajar <pre>Eval avg = Eps = 0.404 Eps = 0.402 Eps = 0.400 Eps = 0.398 Eps = 0.396 Eps = 0.394 Eps = 0.392 Eps = 0.390 Eps = 0.388</pre>

6.	Kinerja Agen	Agen cepat stabil menjaga keseimbangan tiang dalam <300 episode.	Agen butuh >500 episode untuk mendaratkan pesawat dengan baik.
7.	Hasil Test Agen	Agen mampu menjaga keseimbangan tiang lebih lama dari episode ke episode	Agen dapat melakukan pendaratan halus dan stabil di area target.
8.	Kondisi Berhasil	Episode berakhir jika tiang jatuh atau batas langkah maksimum tercapai	Episode berakhir saat pendaratan sukses atau pesawat jatuh.
9.	Hasil Output	Visualisasi menunjukkan tiang semakin stabil, reward meningkat tiap episode. 	Visualisasi menunjukkan lintasan roket semakin halus dan stabil menuju area pendaratan. 
10.	Hasil Grafik	Grafik reward per episode menunjukkan peningkatan cepat dan kemudian mendatar di sekitar nilai maksimum. Ini berarti DQN cepat belajar pola kontrol optimal, karena ruang aksi kecil dan sistem deterministik.	Grafik reward naik secara lambat dan tidak stabil, karena agen butuh waktu untuk memahami kapan harus mengaktifkan thruster kiri, kanan, atau utama. Setelah beberapa ratus episode, reward mulai stabil di sekitar nilai positif, menandakan agen berhasil mendarat dengan aman secara konsisten.



Dari hasil pengamatan pada kedua environment, terlihat bahwa algoritma Deep Q-Network (DQN) memiliki kemampuan adaptasi yang baik dalam proses pembelajaran berbasis *trial and error*. Namun, efektivitas dan kecepatan pembelajarannya sangat bergantung pada tingkat kompleksitas lingkungan yang dihadapi agen.

Pada CartPole-v1, proses belajar berjalan cepat karena sistemnya relatif sederhana dan deterministik. Agen hanya perlu menggerakkan kereta ke kiri atau kanan untuk menjaga tiang tetap seimbang. Grafik reward menunjukkan peningkatan yang tajam di awal pelatihan, lalu stabil di sekitar nilai maksimum. Hal ini membuktikan bahwa DQN mampu dengan cepat menemukan kebijakan optimal hanya dengan sedikit eksplorasi tambahan. Dalam beberapa ratus episode pertama, agen sudah bisa memprediksi aksi terbaik berdasarkan keadaan sistem tanpa banyak kesalahan. Hal ini juga menunjukkan bahwa arsitektur jaringan sederhana dengan dua hidden layer sudah cukup efektif untuk environment dengan ruang aksi yang terbatas.

Berbeda dengan LunarLander-v2, di mana proses pembelajarannya jauh lebih rumit karena lingkungan bersifat nonlinier dan memiliki banyak kemungkinan aksi. Agen tidak hanya perlu mengatur arah pendorong, tetapi juga harus mempertimbangkan kecepatan, posisi, serta sudut rotasi agar pendaratan tetap stabil. Pada tahap awal, nilai reward sangat fluktuatif bahkan cenderung negatif karena agen sering gagal mendarat atau menabrak area target. Namun, setelah sekitar 700–800 episode, grafik reward mulai menunjukkan tren peningkatan yang lebih stabil. Hal ini menandakan bahwa agen mulai memahami hubungan antara aksi dan dampak jangka panjang terhadap reward total — sesuatu yang menjadi inti dari pembelajaran penguatan.

Jika dilihat lebih dalam, stabilitas pelatihan pada LunarLander dipengaruhi

oleh beberapa faktor: (1) nilai *learning rate* yang terlalu besar dapat menyebabkan osilasi reward, (2) proses *epsilon decay* yang terlalu cepat membuat agen berhenti bereksplorasi sebelum menemukan strategi terbaik, dan (3) penggunaan *target network* menjadi penting untuk menstabilkan pembaruan bobot jaringan. Dengan mengatur parameter-parameter tersebut secara hati-hati, DQN dapat mencapai performa yang lebih optimal bahkan di lingkungan yang kompleks.

Secara keseluruhan, hasil pengamatan ini membuktikan bahwa DQN tidak hanya mampu memecahkan masalah kontrol sederhana, tetapi juga bisa diterapkan pada sistem dengan dinamika kompleks. Meskipun waktu pelatihannya lebih lama dan membutuhkan sumber daya komputasi yang lebih besar, hasil akhir menunjukkan bahwa agen mampu beradaptasi dan belajar mengembangkan strategi optimal secara mandiri.

7. Kesimpulan

Berdasarkan praktikum dan analisis yang telah dilakukan, maka dapat diambil kesimpulan yaitu:

1. Algoritma Deep Q-Network (DQN) berhasil diterapkan untuk dua environment berbeda, yaitu CartPole-v1 dan LunarLander-v2, dengan hasil pembelajaran yang menunjukkan pola peningkatan performa seiring bertambahnya episode.
2. Pada CartPole-v1, agen dapat belajar dengan cepat karena lingkungan yang sederhana dan deterministik. Reward meningkat tajam pada awal pelatihan dan stabil di sekitar nilai maksimum setelah ± 300 episode.
3. Pada LunarLander-v2, agen membutuhkan waktu pelatihan lebih lama karena sistemnya lebih kompleks dengan banyak aksi dan kondisi fisik. Reward awal bernilai negatif, namun perlahan meningkat dan mulai stabil setelah ± 800 episode.
4. Perbedaan hasil antara CartPole dan LunarLander menunjukkan bahwa semakin kompleks lingkungan, semakin lama waktu dan iterasi yang dibutuhkan untuk mencapai konvergensi.

8. Saran

- a. Sebaiknya dilakukan penyesuaian parameter pelatihan (hyperparameter tuning) seperti *learning rate* dan *epsilon decay* agar proses pembelajaran lebih cepat dan stabil.
- b. Untuk environment yang lebih kompleks seperti LunarLander, bisa dicoba menggunakan Double DQN agar hasil pembelajaran lebih akurat dan tidak mudah fluktuatif.
- c. Perlu ditambahkan visualisasi hasil simulasi agar perkembangan agen selama proses pelatihan dapat diamati dengan lebih jelas.

9. Daftar Pustaka

- Li, Y. (2018). *Deep Reinforcement Learning: An Overview* (No. arXiv:1701.07274). arXiv. <https://doi.org/10.48550/arXiv.1701.07274>
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. (2013). *Playing Atari with Deep Reinforcement Learning* (No. arXiv:1312.5602). arXiv. <https://doi.org/10.48550/arXiv.1312.5602>
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533. <https://doi.org/10.1038/nature14236>